

Техническое задание

Telegram-бот «Ироничная секретарша»

1. Описание проекта

Нужно создать простого Telegram-бота для ведения личных задач.

Бот должен помогать пользователю:

- добавлять разовые задачи;
- добавлять повторяющиеся ежедневные или периодические задания;
- просматривать список активных задач;
- отмечать задачи выполненными;
- получать условные награды;
- получать короткие ироничные комментарии от виртуальной секретарши.

Проект является учебным. Сложная архитектура, база данных, искусственный интеллект и голосовые команды не требуются.

Все данные должны храниться в CSV-файлах.

2. Технологии

Использовать:

- Python 3.11 или новее;
- библиотеку `aiogram 3`;
- стандартный модуль `csv`;
- библиотеку `APScheduler` для периодических проверок;
- файл `.env` для Telegram-токена.

Запуск программы:

```
python bot.py
```

3. Хранение данных

Использовать три CSV-файла.

3.1. `tasks.csv`

Хранит разовые задачи.

Поля:

```
id
title
deadline
status
created_at
completed_at
reward
```

Пример:

```
id,title,deadline,status,created_at,completed_at,reward
1,Подготовить домашнее задание,2026-07-30 18:00,active,2026-07-29 19:00,,10
```

Статусы:

```
active
completed
cancelled
```

3.2. recurring_tasks.csv

Хранит повторяющиеся задания.

Поля:

```
id
title
period_type
period_value
last_completed
status
reward
```

Возможные значения `period_type`:

```
daily
weekly
interval_days
```

Примеры:

```
id,title,period_type,period_value,last_completed,status,reward
1,Сделать зарядку,daily,1,2026-07-29,active,5
2,Проверить расходы,weekly,monday,2026-07-27,active,8
3,Сделать резервную копию,interval_days,3,2026-07-28,active,10
```

3.3. profile.csv

Хранит данные пользователя.

Поля:

```
telegram_id
name
points
completed_tasks
```

Пример:

```
telegram_id,name,points,completed_tasks
123456789,Пользователь,25,3
```

Если файл отсутствует, бот должен создать его автоматически.

4. Основные команды

Бот должен поддерживать команды:

```
/start
/menu
/add
/add_regular
/tasks
/regular
/done
/profile
/help
```

5. Команда `/start`

При первом запуске бот должен:

1. Проверить наличие CSV-файлов.
2. Создать отсутствующие файлы.
3. Создать профиль пользователя.
4. Показать приветствие.
5. Показать главное меню.

Пример приветствия:

Добро пожаловать. Я ваша виртуальная секретарша.

Я буду хранить задачи, напоминать о делах и иногда комментировать вашу продуктивность. Не волнуйтесь, осуждать буду умеренно.

6. Главное меню

Использовать reply-клавиатуру.

Кнопки:

Добавить задачу
Добавить регулярную
Мои задачи
Регулярные задания
Профиль
Помощь

7. Добавление разовой задачи

Команда:

`/add`

или кнопка:

Добавить задачу

Бот последовательно спрашивает:

1. Название задачи.
2. Дату выполнения.
3. Время выполнения.

Пример:

Что нужно сделать?

Пользователь:

Подготовить презентацию.

Бот:

На какую дату поставить задачу? Введите в формате `30.07.2026`.

Бот:

Во сколько напомнить? Введите в формате `18:00`.

После ввода задача сохраняется в `tasks.csv`.

Ответ:

Задача добавлена.

«Подготовить презентацию» — до 30 июля, 18:00.

Прекрасно. Теперь осталось только действительно её выполнить.

Сложные форматы дат распознавать не требуется.

8. Добавление регулярного задания

Команда:

```
/add_regular
```

Бот спрашивает:

1. Название задания.
2. Тип повторения.
3. Периодичность.

Варианты:

Каждый день
Раз в неделю
Раз в несколько дней

Для ежедневного задания дополнительный параметр не требуется.

Для еженедельного задания выбрать день недели.

Для задания раз в несколько дней ввести число от 2 до 30.

После создания запись сохраняется в `recurring_tasks.csv`.

Пример ответа:

Регулярное задание добавлено: «Сделать зарядку».

Повторение — каждый день.

Ваше тело, возможно, пока об этом не знает, но планы уже серьёзные.

9. Просмотр разовых задач

Команда:

`/tasks`

Бот показывает все активные задачи.

Пример:

Активные задачи:

1. Подготовить домашнее задание
До 30.07.2026, 18:00
2. Проверить сайт
До 31.07.2026, 12:00

Под сообщением добавить inline-кнопки:

Выполнить №1
Выполнить №2

Отменить №1
Отменить №2

Если задач нет:

Активных задач нет.

Либо вы всё сделали, либо ничего не записали. Оба варианта звучат подозрительно.

10. Просмотр регулярных заданий

Команда:

/regular

Бот показывает активные регулярные задания.

Пример:

Регулярные задания:

1. Сделать зарядку
Каждый день
Награда: 5 очков
2. Проверить расходы
Каждый понедельник
Награда: 8 очков

Под каждым заданием должна быть кнопка:

Выполнено №1

11. Выполнение задачи

После нажатия кнопки выполнения бот должен:

1. Найти задачу по ID.
2. Изменить статус на `completed`.
3. Записать дату выполнения.
4. Добавить награду в `profile.csv`.

5. Увеличить число завершённых задач.
6. Отправить сообщение.

Пример:

Задача выполнена.

Получено 10 очков.

Невероятно. Планирование действительно иногда приводит к результату.

Повторное нажатие не должно начислять награду второй раз.

12. Выполнение регулярного задания

После выполнения регулярного задания:

1. Обновить поле `last_completed`.
2. Начислить награду.
3. Не разрешать повторное выполнение в тот же период.

Для ежедневного задания повторное выполнение в тот же день запрещено.

Пример:

Зарядка отмечена как выполненная.

Получено 5 очков.

Один маленький шаг для человека и огромный шаг для вашей статистики.

13. Профиль и награды

Команда:

```
/profile
```

Бот показывает:

Ваш профиль

Очки: 45

Выполнено задач: 6
Уровень: 2

Расчёт уровня:

Уровень 1: 0–49 очков
Уровень 2: 50–99 очков
Уровень 3: 100–199 очков
Уровень 4: 200 очков и больше

При переходе на новый уровень отправить сообщение:

Новый уровень — 2.

Поздравляю. Теперь вы официально продуктивнее, чем были вчера. По крайней мере, по версии CSV-файла.

Дополнительные игровые ресурсы не нужны.

14. Напоминания

Планировщик должен проверять задачи раз в минуту.

Если до дедлайна задачи осталось менее одного часа, бот отправляет одно напоминание:

До задачи «Подготовить домашнее задание» осталось меньше часа.

Самое время перейти от стратегического планирования к паническим действиям.

Чтобы не отправлять повторное напоминание, можно добавить в `tasks.csv` поле:

reminder_sent

Значения:

0
1

Для регулярных заданий бот должен один раз в день, например в 10:00, присылать список актуальных заданий.

Пример:

Регулярные задания на сегодня:

День только начался, а обязанности уже проявили инициативу.

15. Ироничные комментарии

Искусственный интеллект использовать не требуется.

Создать в коде списки готовых фраз.

Пример:

```
SUCCESS_PHRASES = [  
    "Невероятно. Задача действительно выполнена.",  
    "Отличная работа. Даже придраться почти не к чему.",  
    "Ещё одна задача побеждена бюрократическим методом.",  
]
```

Отдельные списки:

```
WELCOME_PHRASES  
TASK_CREATED_PHRASES  
SUCCESS_PHRASES  
EMPTY_LIST_PHRASES  
REMINDER_PHRASES  
LEVEL_UP_PHRASES
```

Фраза выбирается случайным образом через `random.choice()`.

Ирония должна быть доброжелательной.

Запрещены:

- оскорбления;
 - грубость;
 - давление;
 - насмешки над личными качествами пользователя.
-

16. Обработка ошибок

Бот должен корректно обрабатывать:

- неправильную дату;
- неправильное время;

- пустое название;
- отсутствие CSV-файла;
- пустой CSV-файл;
- повторное выполнение;
- неизвестную команду;
- ошибку чтения или записи.

Пример:

Не удалось распознать дату. Используйте формат `30.07.2026`.

При ошибке файл не должен повреждаться.

Для записи рекомендуется:

1. Прочитать все строки.
2. Изменить нужную строку.
3. Перезаписать файл полностью.

17. Ограничения проекта

В проект не нужно добавлять:

- базу данных;
- Docker;
- веб-интерфейс;
- голосовые сообщения;
- нейросеть;
- генерацию изображений;
- карту города;
- сложные достижения;
- регистрацию нескольких пользователей;
- аналитику;
- авторизацию;
- интеграцию с календарём;
- часовые пояса;
- сложную архитектуру.

Бот рассчитан на одного пользователя.

Telegram ID владельца хранится в `.env`:

```
TELEGRAM_OWNER_ID=123456789
```

Сообщения от других пользователей бот должен игнорировать или отвечать:

Этот бот работает в закрытом режиме.

18. Структура проекта

```
simple_secretary_bot/  
├─ bot.py  
├─ config.py  
├─ csv_storage.py  
├─ keyboards.py  
├─ scheduler.py  
├─ phrases.py  
├─ requirements.txt  
├─ .env.example  
├─ tasks.csv  
├─ recurring_tasks.csv  
├─ profile.csv  
└─ README.md
```

Допускается сделать ещё проще и объединить часть модулей.

Минимально допустимо:

```
bot.py  
storage.py  
phrases.py
```

19. Требования к README

В `README.md` описать:

1. Назначение проекта.
2. Установку Python.
3. Создание Telegram-бота через BotFather.
4. Создание `.env`.
5. Установку библиотек.
6. Запуск программы.
7. Список команд.
8. Формат CSV-файлов.

Команды:

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Установка:

```
pip install -r requirements.txt
```

Запуск:

```
python bot.py
```

20. Критерии готовности

Проект считается готовым, если:

- бот запускается;
- доступен только владельцу;
- создаёт CSV-файлы;
- позволяет добавить разовую задачу;
- позволяет добавить регулярное задание;
- показывает список задач;
- позволяет отметить задачу выполненной;
- начисляет очки;
- показывает профиль и уровень;
- отправляет напоминание перед дедлайном;
- отправляет ежедневный список регулярных заданий;
- использует ироничные готовые фразы;
- не начисляет награду повторно;
- после перезапуска данные сохраняются.

21. Промпт для Codex

Создай учебный проект Telegram-бота `simple_secretary_bot` по приложенному техническому заданию.

Требования:

- Python 3.11+;
- aiogram 3;
- APScheduler;
- python-dotenv;
- без базы данных;
- данные хранятся в `tasks.csv`, `recurring_tasks.csv` и `profile.csv`;
- бот рассчитан на одного владельца;

- без голосовых команд;
- без LLM и внешнего AI;
- ироничные ответы выбираются из готовых списков;
- код должен быть простым и понятным начинающему;
- не используй сложные архитектурные шаблоны;
- не добавляй Docker, Redis, PostgreSQL и веб-интерфейс.

Реализуй:

1. `/start` и главное меню.
2. Добавление разовой задачи.
3. Добавление регулярного задания.
4. Просмотр задач.
5. Просмотр регулярных заданий.
6. Выполнение и отмену задачи.
7. Начисление очков.
8. Уровни пользователя.
9. Напоминание за час до дедлайна.
10. Ежедневное сообщение с регулярными заданиями.
11. Защиту от повторной награды.
12. Автоматическое создание CSV-файлов.
13. Проверку даты и времени.
14. Ограничение доступа по `TELEGRAM_OWNER_ID`.
15. `README.md` с инструкцией запуска.

Сначала покажи структуру файлов. Затем создай все файлы проекта полностью.

После создания проверь:

```
python -m compileall .
```

Также проверь, что все импорты указаны в `requirements.txt`.

Не усложняй проект. Код должен запускаться локально после установки зависимостей и добавления Telegram-токена в `.env`.